

A Corrected Metatheory for Mergeable Replicated Data Types

Anonymous Authors

Abstract

Mergeable replicated data types reconcile two branch states relative to a common ancestor. The original bottom-up linearization argument for this model does not hold: a global arbitration relation can change when an event outside the version being linearized absorbs an edge. We give a corrected Lean 4 metatheory based on set-relative arbitration, a reachable store invariant, and a contextual ternary Join lemma. We keep the operational MRDT signature free of client invariants and applicability fields. Datatype implementers instead supply generation, convergence, sequential-refinement, and safety certificates. The framework also supplies canonical virtual LCAs and an asynchronous distributed collector for commit history. Datatype-state garbage collection is optional and composes with the history collector by trace erasure. All theorem names in this paper are checked by the repository's `RefactorLedger.lean` build target.

Contents

1	Result and evidence boundary	2
2	Raw MRDT semantics	2
2.1	The signature stays plain	2
2.2	Configurations and transitions	3
3	Why the earlier induction fails	3
4	Canonical adequacy	4
4.1	Canonical states	4
4.2	The ternary Join obligation	4
5	Generation, sequential meaning, and safety	5
6	Canonical virtual LCAs	6
7	Distributed commit-history garbage collection	7
7.1	Minimal local state	7
7.2	Collection certificate	7
7.3	Direct asynchronous refinement	7
8	Optional datatype-state garbage collection	8
9	Machine-checked instances	9
10	Reproduction and residual boundary	9

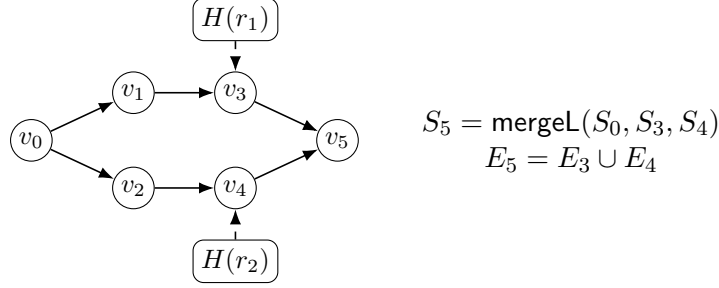


Figure 1: A version-DAG execution. Solid arrows point from parents to children. The merge at v_5 reads the registered LCA v_0 and both branch heads.

1 Result and evidence boundary

An MRDT stores immutable versions in a Git-like directed acyclic graph. An update extends one replica head. A merge of branch states a and b reads the state l at their common ancestor and computes $\text{mergeL}(l, a, b)$. The client expects the state at every registered version to equal a legal sequential fold of exactly that version’s events.

Figure 1 is the running execution shape. The graph is not decoration: apply creates a one-parent version, merge creates a two-parent version, and a later merge may use an old version as its base. This is why the correctness invariant ranges over the store rather than only the current heads.

The current artifact separates four kinds of evidence.

Layer	Framework supplies	Datatype supplies
Raw execution	version DAG, apply, merge, query	state and operations
Convergence	canonical adequacy, virtual LCA	contextual Join proof
Client protocol	guarded execution	generation guard and history bridge
Client meaning	certificate interfaces	sequential and safety proofs
Commit GC	distributed protocol and refinement	none
State GC	generic trace-erasure theorem	optional physical protocol

This boundary matters. A theorem about a semantic MRDT does not by itself verify an editor widget, a wire decoder, or crash recovery. The JavaScript runtime is tested against the model-facing fixtures, but it is not extracted from Lean. We call Lean declarations *machine-checked*, runtime test results *validated*, and benchmark observations *measured*.

2 Raw MRDT semantics

2.1 The signature stays plain

Definition 2.1 (MRDT signature). *An MRDT signature contains a state type Σ , initial state σ_0 , application operation, query interface, conflict-resolution relation, binary merge inherited from the CRDT core, and ternary merge*

$$\text{mergeL} : \Sigma \rightarrow \Sigma \rightarrow \Sigma \rightarrow \Sigma.$$

The compatibility law $\text{mergeL}(\sigma_0, a, b) = \text{merge}(a, b)$ relates the ternary and initial-slice binary operations.

MRDTSig

$$\begin{array}{c}
\frac{H(r) = v \quad V(v) = (S, E) \quad t \notin \text{times}(C) \quad v' \notin \text{dom}(V) \quad v < v' \quad e = (t, r, o)}{C \xrightarrow{\text{apply}(t, r, o)} C[v' \mapsto (\text{do}(S, e), E \cup \{e\}); r \mapsto v'; P(v') \mapsto [v]]} \text{APPLY} \\
\\
\frac{H(r_1) = v_1 \quad H(r_2) = v_2 \quad \text{LCA}_P(v_1, v_2) = v_T \quad V(v_i) = (S_i, E_i) \ (i \in \{1, 2, T\}) \quad v_m \notin \text{dom}(V) \quad v_1, v_2 < v_m}{C \xrightarrow{\text{merge}(r_1, r_2)} C[v_m \mapsto (\text{mergeL}(S_T, S_1, S_2), E_1 \cup E_2); r_1 \mapsto v_m; P(v_m) \mapsto [v_1, v_2]]} \text{MERGE}
\end{array}$$

Figure 2: Paper-level raw operational semantics. The Lean rules also update the coherent replica caches and extend visibility on apply.

The signature has no fields named invariant or applicable. A client can invoke the raw update on every state. Generation restrictions enter only in the certified execution of Section 5. This choice prevents the datatype carrier, client protocol, and safety theorem from becoming one circular definition.

The optional `LocalCommutationDomain` is a proof aid for algorithms whose updates commute only under local hypotheses. It does not restrict raw execution and is not required by `VerifiedMRDT`.

2.2 Configurations and transitions

A paper-level configuration is

$$C = \langle V, H, P, \text{vis} \rangle, \quad V(v) = (S_v, E_v),$$

where V is the finite version store, H maps replicas to heads, P maps a version to its parents, and vis is event visibility. Replica state and replica events are derived as $S_{H(r)}$ and $E_{H(r)}$. The Lean `Configuration` also caches these two projections as `N` and `L`; `head_coherent` requires the caches to equal the derivation above. We omit the caches from the rules because they contain no independent semantic information.

Version numbers increase along parent edges. The configuration requires immutable event timestamps, per-replica causal order, coherent heads, and the key LCA equation: if v_T is an LCA of v_1 and v_2 , then

$$\mathcal{E}(v_T) = \mathcal{E}(v_1) \cap \mathcal{E}(v_2).$$

Figure 2 gives the state-changing core of `Step`. The notation $C[v \mapsto (S, E); r \mapsto v; P(v) \mapsto \pi]$ updates the version store, replica head, and parent list while preserving all other entries. `QUERY` returns `query( $S_{H(r)}$ ,  $q$ )` without changing C ; `CREATE` installs the initial version at a fresh replica. The raw `APPLY` rule deliberately does not check a datatype-specific client guard.

3 Why the earlier induction fails

The conflict relation induces an arbitration order `lo`. The first proof attempt used a relation computed from the entire configuration while linearizing a smaller version event set. That is unsound when a third event, outside the smaller set, absorbs an arbitration edge.

Proposition 3.1 (global-order convergence is false). *There is a two-operation add-wins signature and a legal configuration with a backward-closed event set E such that two enumerations of E both respect the configuration-global arbitration relation but fold to different states.*
`convergence_over_backward_closed_subsets_false`

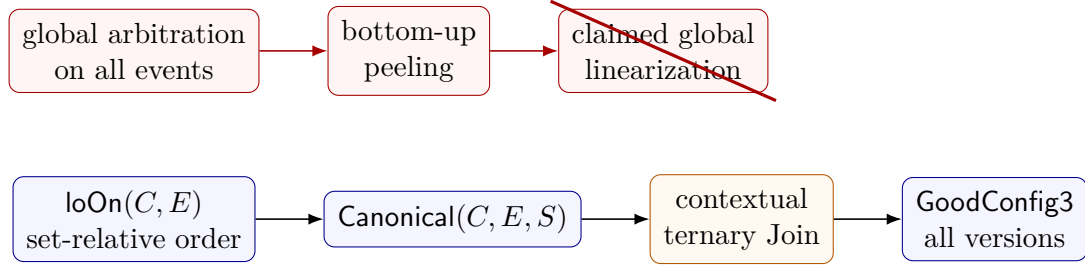


Figure 3: The failed bottom-up argument and the checked canonical-adequacy argument. Red nodes describe the refuted route; blue/orange nodes are current Lean interfaces and invariants.

The checked countermodel is in `Metatheory/Join/Convergence_CounterModel.lean`. The repaired relation `loOn C E` computes arbitration relative to E . In the countermodel it retains the edge that the outside absorber erased globally.

Associativity, commutativity, and idempotence of merge do not repair the proof. The second checked countermodel satisfies the core update laws and a bounded join-semilattice merge but violates both the peel laws and the Join lemma. `coreVCs_lattice_insufficient`

These countermodels establish two boundaries: arbitration must be local to the set being linearized, and convergence needs a contextual bridge between merge and update history.

Figure 3 summarizes the repair. The failed path tried to peel operations from a merge using a global order. The checked path instead proves canonicity for each stored event set and uses one contextual Join obligation at every merge.

4 Canonical adequacy

4.1 Canonical states

For configuration C and event set E , a state is canonical when some duplicate-free enumeration of exactly E respects `loOn C E` and folds from σ_0 to that state. The update-layer lemmas prove that any two such enumerations have the same fold under the supplied update conditions.

The reachable invariant `GoodConfig3` states that every registered version contains a canonical state of its event set. It also retains visibility transitivity and irreflexivity, event support, and causal closure. The invariant ranges over all versions, not only replica heads, because later merges may read an older version as their LCA.

4.2 The ternary Join obligation

Definition 4.1 (contextual ternary Join). *For a configuration C , suppose E_1 and E_2 satisfy the required closure conditions. If s_0 , s_1 , and s_2 are canonical for $E_1 \cap E_2$, E_1 , and E_2 , respectively, then*

$$\text{mergeL}(s_0, s_1, s_2)$$

must be canonical for $E_1 \cup E_2$.

JoinLemma3At

`JoinLemma3` quantifies this obligation over all configurations. Some datatypes prove the weaker per-configuration form from a history fact. The full-closure sibling `JoinLemma3F` supports algorithms whose merge proof needs complete causal closure rather than only non-commuting predecessors.

Theorem 4.2 (ordinary adequacy). *If a datatype satisfies `JoinLemma3`, every configuration reachable from `initConfig` by `Step` is RA-linearizable at every registered version. `ra_linearizable3_of_join`*

The proof carries `StoreInv` with `GoodConfig3`. The store invariant proves that parent reachability remains well founded and that the selected LCA contains exactly the event-set intersection (`lca_events_of_storeInv`). `Apply` extends a canonical fold by a fresh event. `Merge` invokes the `Join` obligation. `Query` and `replica creation` preserve the invariant directly.

The framework retains the eight-condition route as one constructor for `Join`: `CoreVCs3CD`, `FeasibleDeltaVCs3`, and `CDVC3` imply `JoinLemma3`. Datatypes may instead prove `Join` directly; the public `VerifiedMRDT` package depends on the convergence result, not on one particular VC decomposition.

5 Generation, sequential meaning, and safety

RGA insertion illustrates why a plain signature still needs a client protocol. An insertion carries an anchor and a fresh identifier. Raw delivery can apply that record anywhere, but honest minting requires the issuer to have seen the anchor and to choose an identifier above its causal past. This is a historical fact about the minting state, not a shape predicate on an arbitrary later delivery state.

Definition 5.1 (generation contract). *A `GenerationContract` D contains*

$$\begin{aligned} \text{Guard} & : \text{Op} \rightarrow \Sigma \rightarrow \text{Prop}, \\ \text{History} & : \text{Configuration} \rightarrow \text{Prop}, \\ \text{history_of_mint} & : \text{MintHonest}(\text{Guard}, C) \rightarrow \text{History}(C). \end{aligned}$$

`GuardedStep` checks the guard only for an apply transition and checks it at the issuer's current materialized head. Non-apply transitions retain their raw semantics. A final configuration does not store all former issuer heads, so `MintCertifiedReach` explicitly carries a causal-fold mint witness at each step. `MintCertifiedReachV` is the virtual-LCA counterpart.

Definition 5.2 (verified MRDT). *An implementer supplies:*

- a `GenerationContract`;
- a `ConvergenceCertificate` for ordinary and virtual-LCA certified executions;
- an independent `SequentialSpec` and `SequentialRefinement`;
- a bridge from `LinearMintHistory` to the sequential honesty predicate; and
- a `SafetyCertificate` with a state invariant, its preservation on ordinary and virtual certified executions, and an observable consequence.

VerifiedMRDT

The package exposes three capstones: `VerifiedMRDT.converges`, `VerifiedMRDT.convergesV`, and `VerifiedMRDT.sequentially_correct`. The safety certificate remains separate because convergence does not imply a client invariant. For example, the bounded counter proves non-negative observable values through a global history invariant; arbitrary ternary tuples do not preserve that invariant.

$$\begin{array}{c}
\frac{C \xrightarrow{\text{apply}(t,r,o)} C' \quad H(r) = v \quad V(v) = (S, E) \quad G.\text{Guard}((t, r, o), S)}{C \xrightarrow[G]{\text{apply}(t,r,o)} C'} \text{CERTIFIED-APPLY} \\
\\
\frac{C \xrightarrow{\ell} C' \quad \ell \neq \text{apply}(-, -, -)}{C \xrightarrow[G]{\ell} C'} \text{CERTIFIED-OTHER}
\end{array}$$

Figure 4: Generation-certified execution. Certification restricts minting at the issuer head; it does not alter raw delivery or merge semantics. A `MintCertifiedReach` trace additionally retains the historical mint witness needed after the issuer moves to a later head.

6 Canonical virtual LCAs

Criss-cross histories can have several maximal common ancestors and no unique registered LCA. Choosing one maximal ancestor can lose events represented by another. The framework instead constructs a synthetic state by recursively folding the maximal-common-ancestor frontier.

`canonicalVirtualLCA` implements this resolver. The support decreases under the version order, which justifies the recursion. The key theorem `virtualLCASState_canonical` proves that the synthetic state is canonical for the union of the maximal ancestors' event sets. The store theorem `mca_events_cover` shows that this union covers every common ancestor.

`StepV` conservatively embeds every ordinary `Step` and adds one virtual merge constructor. It allocates only the resulting version; the synthetic base is ghost semantic state.

Theorem 6.1 (virtual-LCA adequacy). *The same `JoinLemma3` that proves ordinary adequacy proves RA-linearizability for configurations reachable under `StepV` with the canonical resolver.*
`ra_linearizable3V_of_join`

No new datatype merge law is introduced by virtual LCAs.

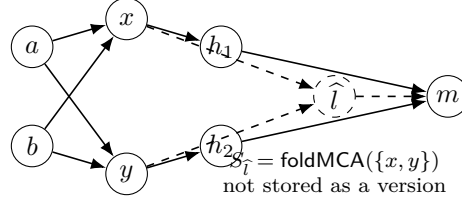


Figure 5: A criss-cross history has two maximal common ancestors, x and y . The canonical virtual LCA is ghost state. Only the merge result m is allocated.

7 Distributed commit-history garbage collection

Commit-history GC belongs to the runtime framework. It is not supplied once per datatype, and the public framework has no global stop-the-world collector.

7.1 Minimal local state

For each replica, the logical collector stores only

$$L = \{\text{head} : \text{Version}, \quad \text{commits} : \mathcal{P}(\text{Version})\}, \quad \text{head} \in \text{commits}.$$

The parent relation, fixed roster, and immutable optional author of each operation commit are protocol parameters. Roots and merge commits are unauthored. An envelope contains only a commit set. Fetch is set union and does not change the receiving head; head synchronization is a separate transition.

GC.Local, GC.Envelope

Evidence is derived rather than stored. Replica r has frontier evidence at L when L retains a commit authored by r that reaches $L.\text{head}$. GC.EvidenceComplete requires such evidence for every other roster member. The negative control GC.EvidenceSPOT.missing_author confirms that an unauthored merge head cannot stand in for a missing author's evidence.

7.2 Collection certificate

A GC.Certificate supplies a keep set containing the local head and the evidence witnesses. It proves support in the local holding and exact reachability and LCA answers between retained versions. The canonical constructor GC.Certificate.ofMCAClosed uses a keep set closed under maximal common ancestors.

The compressed graph connects two retained versions when the original graph reached between them. It need not retain intermediate paths or the old root. GC.compressedReaches_iff proves reachability equivalence, and GC.root_absent_when_dropped proves that a root outside the keep set is absent from the retained carrier.

7.3 Direct asynchronous refinement

The collecting protocol has fetch, head synchronization, and local GC. Its no-GC counterpart has the same network steps and no collection transition. The simulation relates one full and one compact local store by equal heads and commit-set inclusion.

Theorem 7.1 (distributed collection refines no-GC). *Every finite collecting execution is matched by a finite no-GC execution. Fetch and head synchronization match their counterparts; col-*

$$\begin{array}{c}
\frac{M = \text{commits}(W(s))}{W \longrightarrow W[d \mapsto \langle \text{head}(W(d)), \text{commits}(W(d)) \cup M \rangle]} \text{FETCH} \\
\frac{v \in \text{commits}(W(r))}{W \longrightarrow W[r \mapsto \langle v, \text{commits}(W(r)) \rangle]} \text{HEAD-SYNC} \quad \frac{\text{Certificate}(W(r), K)}{W \longrightarrow W[r \mapsto \langle \text{head}(W(r)), K \rangle]} \text{COLLECT}
\end{array}$$

Figure 6: Asynchronous commit-history GC. Fetch is the ordinary remote-fetch path and unions immutable commits. Head synchronization is explicit. Collect is local and requires complete frontier evidence plus reachability/LCA preservation for the keep set.

lection stutters. The final worlds have equal replica heads, so head-state reads are unchanged.
GC.execution_refines_noGC, GC.read_preserved

The runtime lifting adds a ghost MRDT configuration to state the semantic simulation. Visible apply and merge labels evolve that configuration. Fetch and GC remain silent. Erasing silent labels yields an ordinary or virtual-LCA core execution (*GC.runtime_refines_core*, *GC.runtime_refines_coreV*). The ghost configuration is not a second physical copy required of an implementation.

8 Optional datatype-state garbage collection

Commit GC removes versions from a replica’s history. It cannot decide whether an RGA tombstone, a Peritext mark boundary, or a TreeMove event can be removed from the state stored at a retained version. That decision is datatype specific.

Definition 8.1 (state-GC protocol). *A **StateGCProtocol** supplies a physical state, a projection to a semantic configuration, a validity predicate, and physical transitions labelled by an optional semantic label. It proves:*

1. *validity is invariant;*
2. *a silent physical transition leaves the semantic projection unchanged;*
3. *a visible transition refines a **StepV** transition.*

Theorem 8.2 (state collection refines semantic execution). *Erasing silent labels from any valid finite physical trace yields a finite **StepV** trace between the projected configurations.*
StateGCProtocol.refines

The combined physical state contains the datatype protocol state and the generic distributed commit stores. A combined transition is either a datatype physical step or a silent fetch/commit-GC step.

Theorem 8.3 (the two collectors compose). *For every valid finite combined trace, erasing state-GC, fetch, and commit-GC labels yields an execution of the uncollected virtual-LCA semantics.*
GC.CombinedSteps.refinesV

The ordinary corollary *GC.CombinedSteps.refinesRaw* applies when every visible datatype transition proves a raw **Step**. Sided Peritext and TreeMove provide concrete state-GC protocols. Other datatypes may omit the interface entirely.

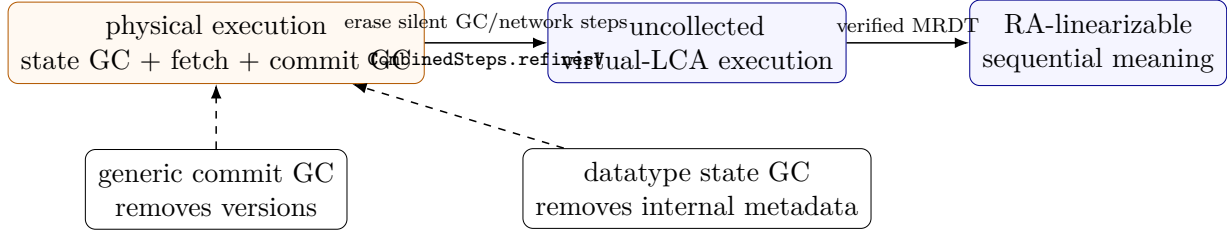


Figure 7: The two collectors remove different objects and compose by trace erasure. Neither collector changes the visible MRDT semantics.

9 Machine-checked instances

Table 1 lists the packages imported by `Metatheory/RefactorLedger.lean`. A checkmark means that the current Lean tree contains the named public package; it does not claim that the JavaScript runtime is extracted from that package.

Datatype	Generation	Convergence	Sequential	State GC
Bounded counter	✓	✓	✓	–
RGA with tombstones	✓	✓	✓	–
EmbedRGA	✓	✓	✓	policy lemmas
SidedEmbedRGA/FugueMax	✓	✓	✓	policy lemmas
MVR	✓	✓	✓	–
Mergeable queue	✓	✓	✓	–
Peritext	✓	✓	✓	concrete protocol
TreeMove	✓	✓	✓	concrete protocol

Table 1: Current nontrivial certificate packages. Trivial-generation grow-only and counter products are also in the ledger.

The production ledger also checks product construction, Fugue and FugueMax non-interleaving results, Peritext rendering, the distributed collector, and the combined GC theorem. It is an import-and-name gate rather than an axiom eliminator. The cited instance files print standard Lean axioms only; the repository release script separately rejects `sorry` and `sorryAx` in the production tree.

10 Reproduction and residual boundary

Run the production proof and runtime gate from the repository root:

```
./scripts/check-mrtd-refactor.sh
```

Run the two paper builds with:

```
./scripts/check-working-papers.sh
```

The proved framework covers semantic MRDT executions, canonical virtual LCAs, distributed commit-history collection, optional state-GC protocols, and their composition. Same-replica crash recovery is not yet part of the verified or tested continuation protocol. The JavaScript implementation uses indexes and cached metadata for efficiency; the paper-facing logical GC state remains the minimal `{head, commits}` record.